AI ACCELERATOR ARCHITECTURE FOR SECOND MODEL

Sparsely Embedded Convolutional Detection — Hardware-Aware LiDAR 3D Object Detection under Adverse Weather

Mid Term Project Evaluation

by

Yash Agrawal

Master of Technology of Integrated Circuit Design and Technology
Atal Bihari Vajpayee-Indian Institute of Information Technology and Management, Gwalior

Department

Electrical and Electronics Engineering

Supervisor

Prof. Manisha Pattanaik

Dated: 16 October 2026

LiDAR point clouds → Fog-physics augmentation → SECOND Model profiling → Syncing With Appropriate Board → Implementation

Objective & Scope

- Execute the trained, validated and tested SECOND Model On FPGA Board— the second AI model for 3D Object Detection
- Analyze SECOND's computation layer-by-layer: sparse 3D convolution backbone, BEV(Bird Eye View) reshape, and 2D RPN (region Proposal Network) detection head.
- Identify computational and memory requirements to guide hardware mapping and dataflow design.
- Define the complete implementation path through RTL, and Implement it On Appropriate Board (Probably, Genesys 2)
- Study robustness of the target workload on fog-degraded point clouds, to keep the Model design weather-aware

Working title: Hardware Accelerator Architecture for SECOND-Model Based LiDAR 3D Object Detection under Adverse Weather

MODEL SELECTION RATIONALE

Why SECOND Model for Hardware Acceleration?

Regular compute pattern

One-stage, voxel-based detector — no irregular point-based refinement stage, unlike two-stage models (e.g., PV-RCNN).

Sparse conv is dominant

Sparse 3D convolution is the single largest compute block — a well-studied, well-bounded target for custom dataflow.

Proven accuracy/efficiency

Established KITTI baseline (Car/Pedestrian/Cyclist, easy/moderate/hard) — gives a credible reference point.

Right scope

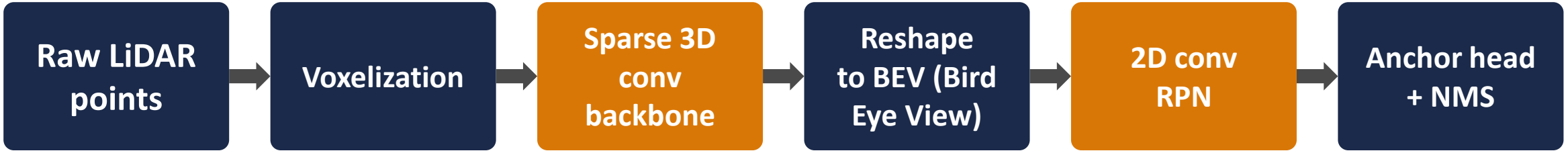
Lower architectural complexity than PV-RCNN++ — feasible to carry through RTL and Implementation on Appropriate Board (Probably, Genesys 2) in this timeline.

THIS WEEK'S READING

Literature & Codebase At a Glance

Source	Focus	Relevance to this project
Shinde et al., 2024	Fog particle-size distribution (Gamma/Junge) effects on 3D detection accuracy	Shows how droplet-size modeling changes attenuation/backscatter and class-wise accuracy
Liu et al., 2022 (Parallel LiDARs)	Mie-theory-based attenuation/backscatter, evaluated on PointPillars, SECOND, PointRCNN	Directly evaluates SECOND under four fog types — a useful accuracy reference
Hahner et al. (paper + GitHub repo)	Physics-based fog simulation on real LiDAR point clouds	Source of the simulation pipeline used this week to generate fog-augmented point clouds

SECOND Model: Pipeline & Hardware-Critical Operations



Orange = the two most hardware-critical, compute-heavy stages

- Sparse 3D convolution (submanifold + regular) — dominant MAC count, irregular active-voxel addressing.
- Voxel feature encoding — per-voxel point aggregation before the backbone.
- Dense 2D convolution in the *Region Proposal Network* **(RPN)** over the BEV feature map.
- Anchor-based detection head and *Non-Max Suppression* **(NMS).**

PHYSICS STUDIED THIS WEEK

Fog Simulation: What Was Understood

Core relations

T(R) = exp(−αR)
One-way transmission loss over range R

Hard-target return ≈ exp(−2αR) × P_clear(R)
α, β obtained by integrating Mie efficiencies over droplet-size distribution N(D)

- α (attenuation) governs point loss / signal decay with range.
- β (backscatter) governs false / soft-target returns — the visual “fog noise.”
- Gamma and Junge distributions model droplet-size N(D) for different fog severities.

Hahner LiDARfogsim repository

- fog_simulation.py — augments clear-weather point clouds with the physics above
- theory.py — single-beam fog return model, matched against the paper equations
- Precomputed integral lookup tables for α, β — avoids re-solving the Mie integrals at runtime

RISK AWARENESS

Challenges Faced till now

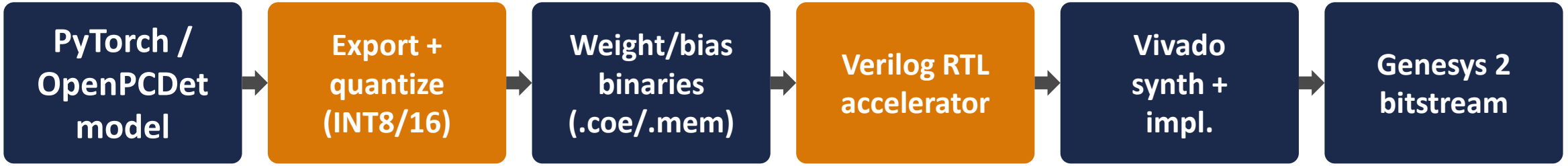
Challenge	Mitigation / next step
Setting up the sparse-convolution / point-cloud toolchain (spconv, Open3D, CUDA) on Ubuntu	Pin package versions; document a reproducible install script
Interpreting the Mie-theory attenuation/backscatter code and matching it to the paper equations	Cross-check theory.py output against the values from the papers
Selecting fog severities (α, β) that meaningfully stress SECOND's sparsity pattern	Sweep severity levels and inspect active-voxel counts before committing to a test matrix
Implement The Model On Genesys 2 Board Because Of The Not So Familiar Compatibility With SECOND Model Python Code	Use AMD Vivado As a bridge To Fill The Compatibility Issue Gap, And, Use Divide and Conquer Approach To Implement SECOND Model on Genesys 2

FPGA Choice: Genesys 2 (Kintex-7)

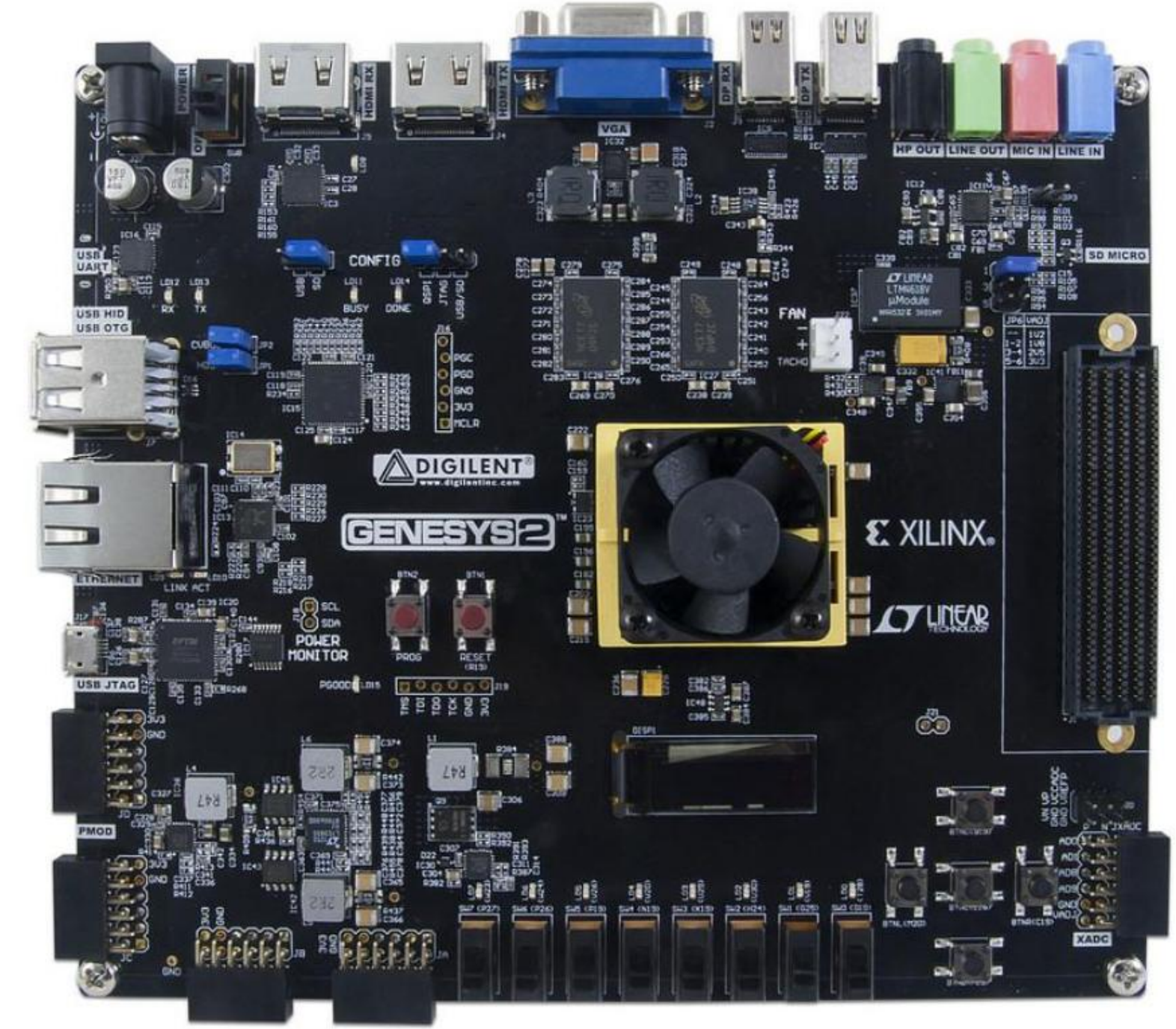
Resource	Genesys 2 (XC7K325T-2FFG900C)
Logic slices	50,950 (4× 6-input LUT + 8 FF each)
DSP slices	840
Block RAM	≈ 16 Mb
Max internal clock	> 450 MHz
Onboard memory	1 GB DDR3 (1800 Mt/s)
Host link options	USB-JTAG/UART, Gigabit Ethernet, FMC HPC

- 840 DSP slices give a strong base for a MAC-heavy PE array (sparse + dense convolution).
- Ample BRAM supports on-chip double-buffering of voxel/feature maps to cut external memory traffic.

From Trained Model to FPGA



Host ⇄ FPGA runtime loop: Python on host PC streams a test point cloud / voxel grid to the board over UART or Gigabit Ethernet; the accelerator returns detections for comparison and visualization.

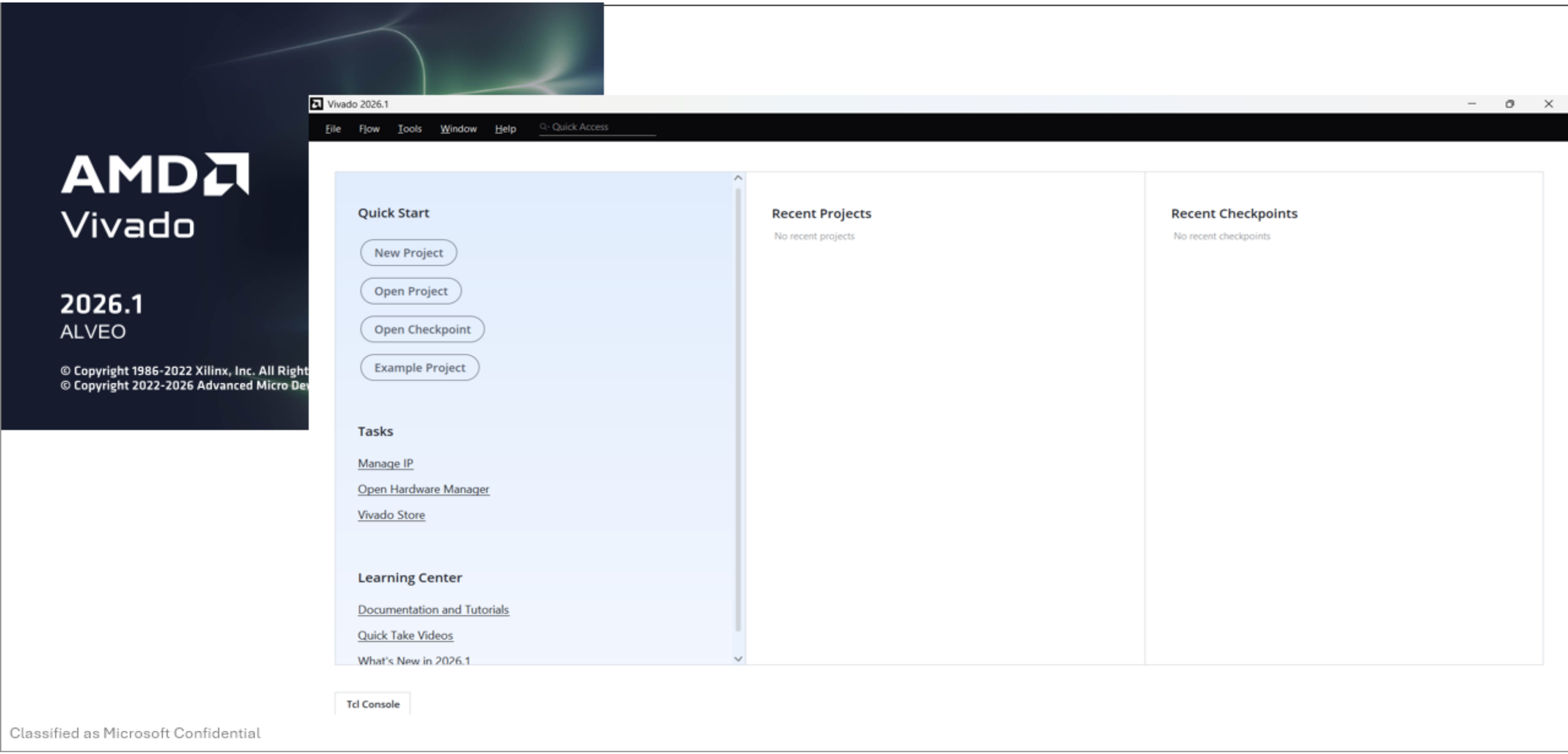


Genesys 2
Board's
Bird Eye View



Genesys 2
Board
Side View

AMD VIVADO



Classified as Microsoft Confidential

Software–Hardware Co-Verification

1. Generate golden reference vectors (layer activations + final output) from the PyTorch model on a fixed test input.

2. Quantize the same input and feed it into the Verilog testbench in simulation (Vivado Simulator); compare against the golden vectors, layer by layer.

3. Only after simulation matches, move to on-board bring-up — load the bitstream and stream the same test vectors from the host over UART/Ethernet.

4. Compare FPGA-captured output to the Python reference within a defined numerical tolerance before trusting any hardware result.

LOOKING AHEAD

Plan for the Future

1. Implement trained (On OpenPCDet's KITTI dataset), validated and tested- SECOND model (in clear wheather) on Genesys 2 Board

2. Generate a broader set of fog-augmented samples across multiple severity levels.

3. Begin layer-wise profiling of SECOND Model: active-voxel counts, operation counts, tensor shapes per layer.

4. Implement a trained, validated and tested- SECOND model (in adverse wheather) on Programmed Genesys 2 Board

- Model weights live in a PyTorch .pth file, and the Genesys 2 can't read that format.
- So, Weights would be exported to plain numbers, quantize them, and load them into the board's memory.
- SECOND checkpoints are a few hundred MB. SECOND has about 5 million parameters, so the file should be much smaller, roughly 20 MB for weights alone and a few times that with optimizer state included.
- **The Genesys 2 can't run your Python.** It has a Kintex-7 FPGA with no ARM CPU, so Python and PyTorch stay on your PC. The board runs hardware you design in RTL, HLS, or a MicroBlaze soft core, and your PC talks to it over UART or Ethernet.
- **The full SECOND model won't fit.** Its sparse 3D convolutions (spconv) and voxelization don't map well to this FPGA, which has about 2 MB of block RAM, 840 DSPs, and 1 GB of DDR3. The realistic target is the dense part, the 2D BEV backbone and detection head, with the rest on the PC.
- **Step 1: Export the weights from the checkpoint**
- **Step 2: Quantize to int8 and write memory-init files:** Before quantizing, fold each BatchNorm into its preceding conv. That removes extra hardware, and it's standard practice.
- **Step 3: Get the weights onto the board**
- **Small layers:** initialize BRAM directly from the .mem or .coe file in Vivado ($readmemh in RTL, or the Block Memory Generator's .coe option).
- **Larger layers:** to be stored in the 1 GB DDR3 and load them from the SD card at boot, or send them over UART or Ethernet through a MicroBlaze.
- **Checking:** run the same input through PyTorch and the board, then compare outputs layer by layer.
- **Which toolflow is being used on the board:** hand-written Verilog/VHDL. The loading method and the weight layout both depend on it, so the code is different for that path.

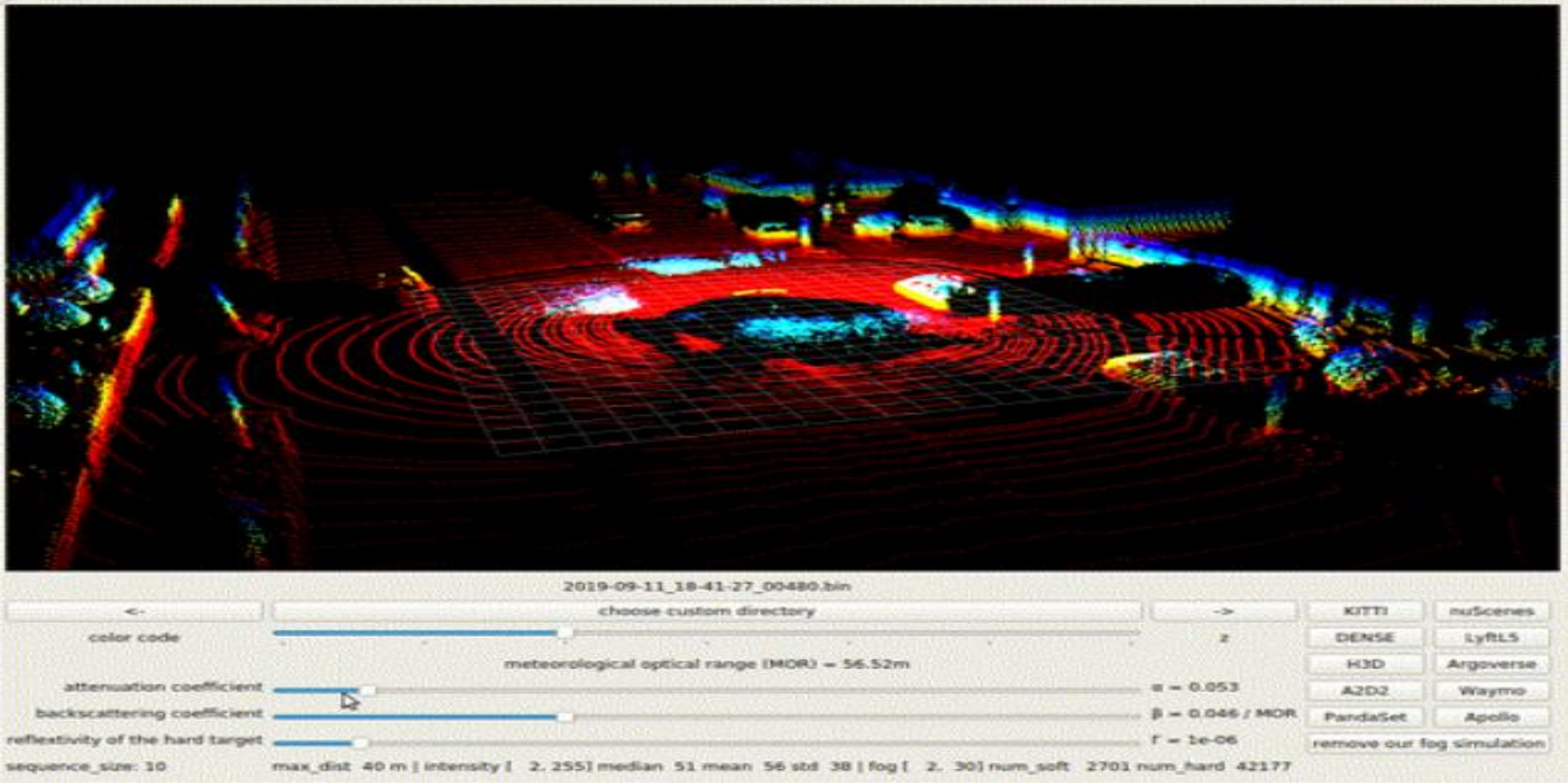
STATUS

Work Completed This Now

- ✓ Read and summarized 3 papers on fog-particle physics and its effect on LiDAR 3D detection.
- ✓ Studied Martin Hahner's LiDARfogsim GitHub repository — simulation entry point, theory implementation, lookup tables.
- ✓ Set up the Python / Ubuntu environment (previously named foggy_lidar -> yash_ai_accelerator) and generated fog-augmented point clouds from clear-weather LiDAR scans.
- ✓ Mapped The Trained SECOND Model’s operator pipeline to identify the hardware-critical stages for accelerator design.

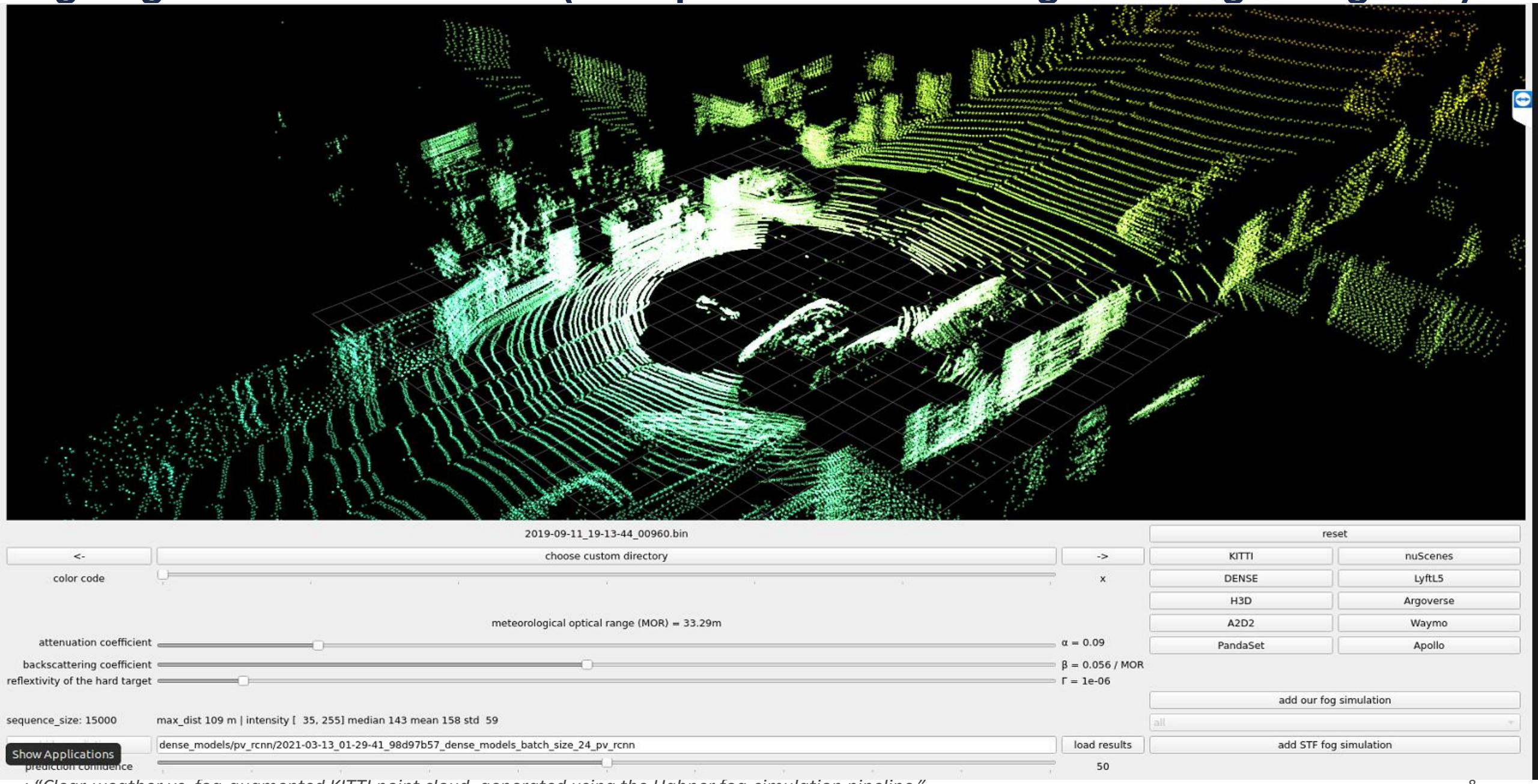
Not yet completed: layer-wise profiling, RTL work.

Fog-Augmented Point Clouds (Example Point Cloud Image with fog)



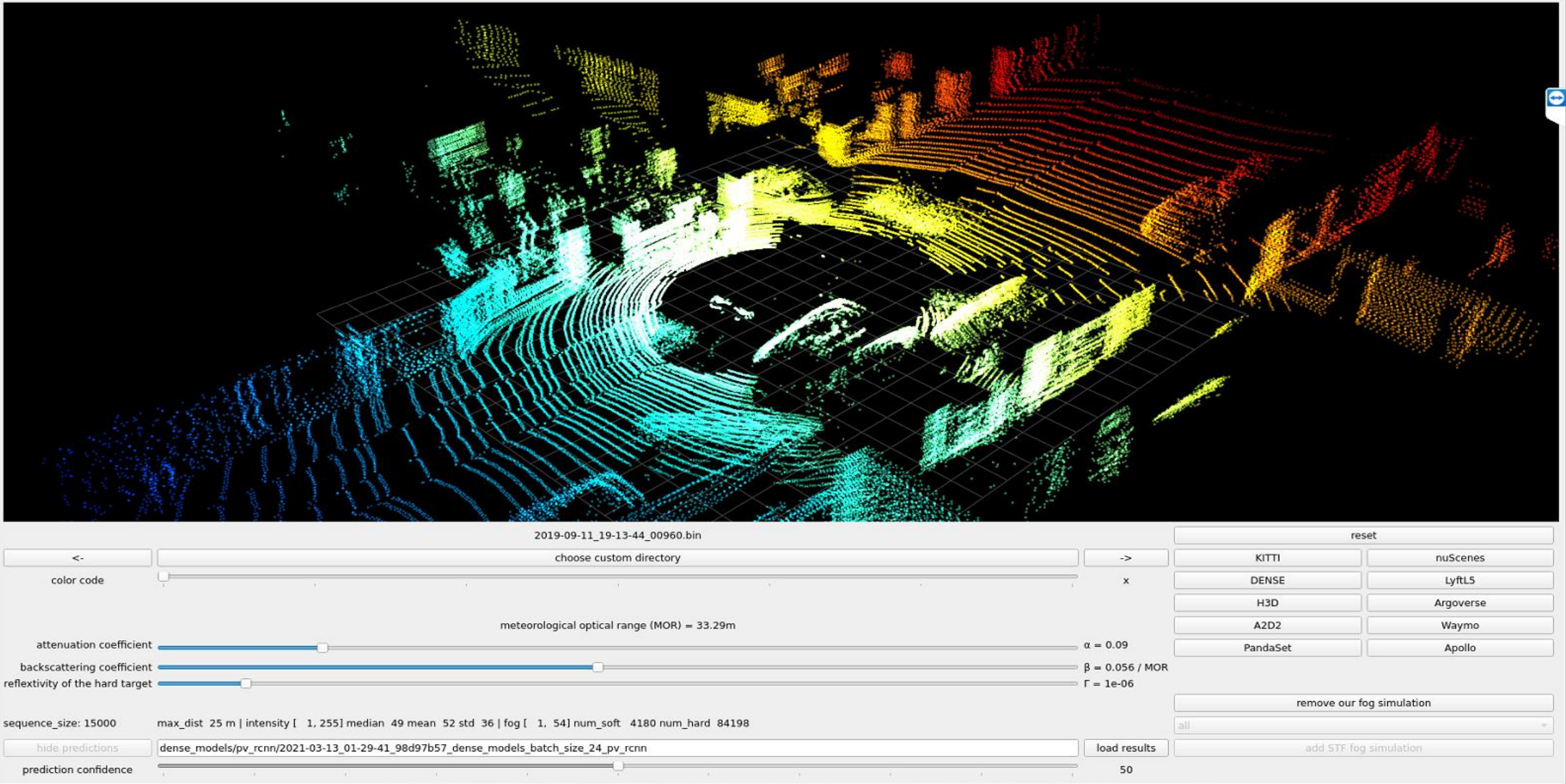
: "Clear-weather vs. fog-augmented KITTI point cloud, generated using the Hahner fog-simulation pipeline."

Fog-Augmented Point Clouds (Example Point Cloud Image with fog In negative)



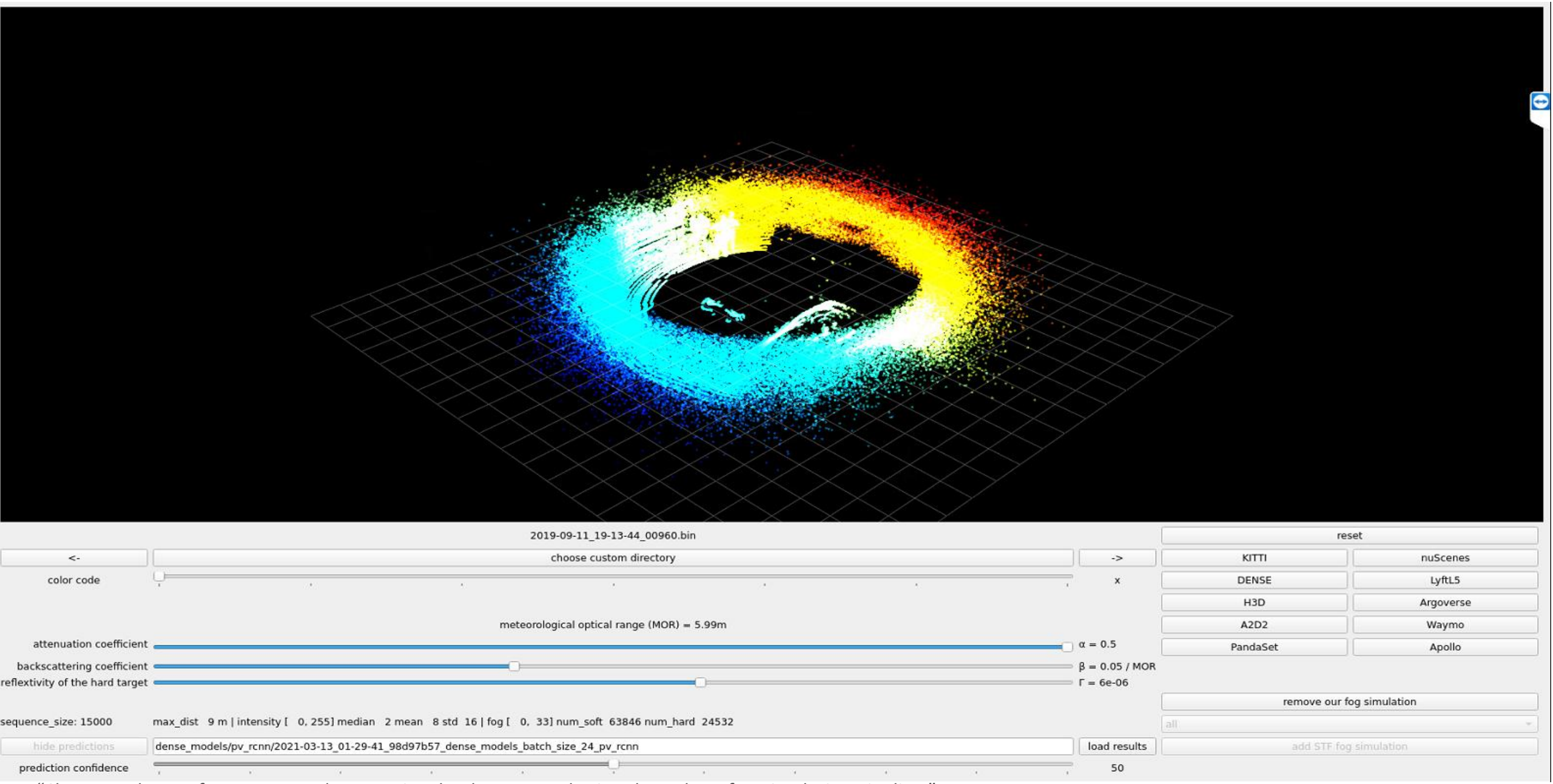
: *"Clear-weather vs. fog-augmented KITTI point cloud, generated using the Hahner fog-simulation pipeline."*

Fog-Augmented Point Clouds (Example Point Cloud Image with fog from different angle)



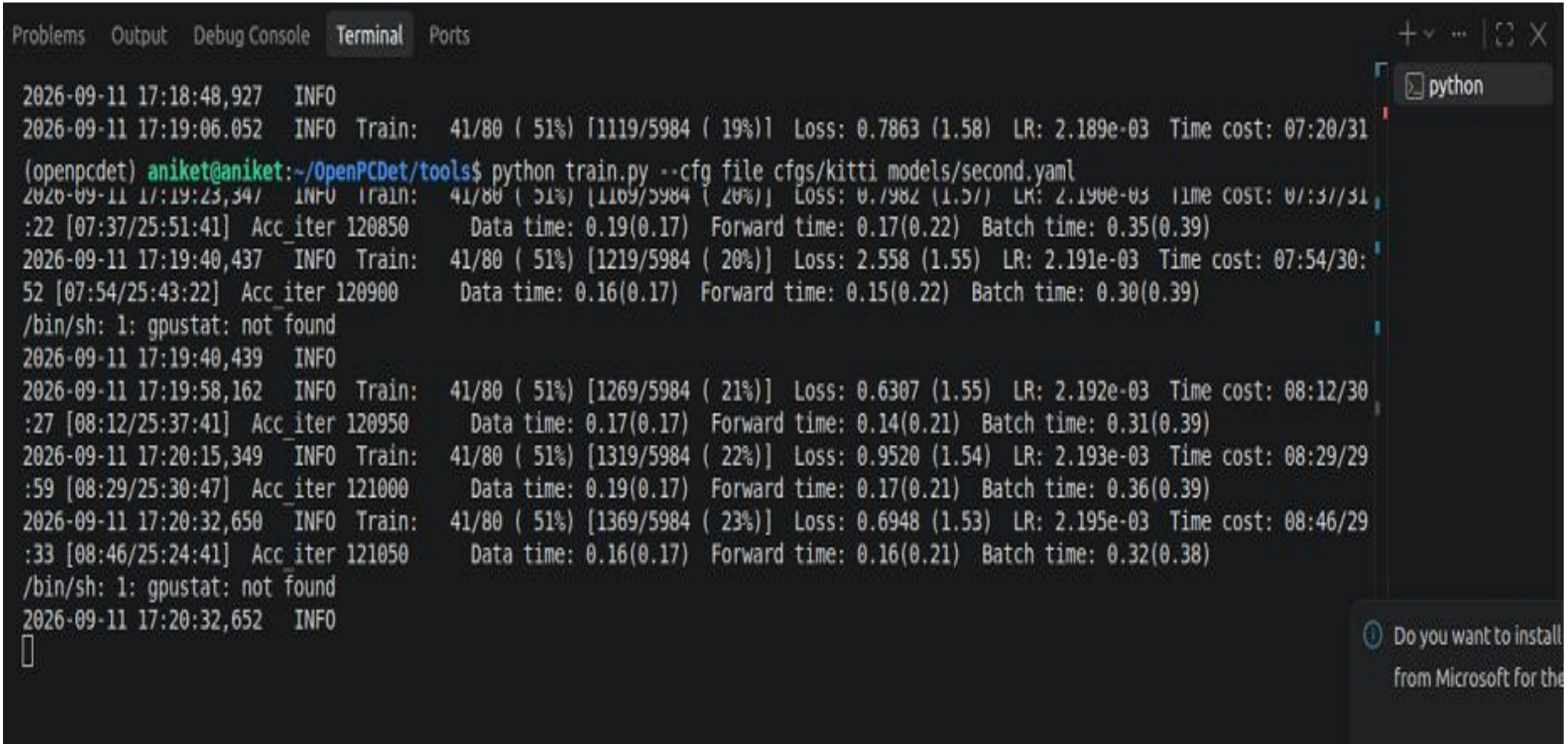
: "Clear-weather vs. fog-augmented KITTI point cloud, generated using the Hahner fog-simulation pipeline."

Fog-Augmented Point Clouds (Example Point Cloud Image with very dense fog, Martin Hahner)



: "Clear-weather vs. fog-augmented KITTI point cloud, generated using the Hahner fog-simulation pipeline."

Training Of Second Model (on Clear Wheather) To Be Mapped For Hardware Acceleration On FPGA Board (Probably Genesys2)



```
(openpcdet) aniket@aniket:~/OpenPCDet/tools$ cat cfgs/kitti_models/second.yaml
        NMS_PRE_MAXSIZE: 4096
        NMS_POST_MAXSIZE: 500


OPTIMIZATION:
    BATCH_SIZE_PER_GPU: 1
    NUM_EPOCHS: 80

    OPTIMIZER: adam_onecycle
    LR: 0.003
    WEIGHT_DECAY: 0.01
    MOMENTUM: 0.9

    MOMS: [0.95, 0.85]
    PCT_START: 0.4
    DIV_FACTOR: 10
    DECAY_STEP_LIST: [35, 45]
    LR_DECAY: 0.1
    LR_CLIP: 0.0000001
```

Second model configuration file

The Model Was fully trained Till Epoch 80 (please See right image), for results , PTO

```
(openpcdet) aniket@aniket:~/OpenPCDet/tools$ python test.py --cfg_file cfgs/kitti_models/second.yaml --ckpt ../output/kitti_models/second/default/ckpt/checkpoint_epoch_80.pth --batch_size 1
g argument. (Triggered internally at ../aten/src/ATen/native/TensorShape.cpp:3526.)
  return _VF.meshgrid(tensors, **kwargs)  # type: ignore[attr-defined]
2026-10-07 18:42:43,981   INFO  ==> Loading parameters from checkpoint ../output/kitti_models/second/default/ckpt/checkpoint_epoch_80.pth to GPU
2026-10-07 18:42:44,217   INFO  ==> Checkpoint trained from version: pcdet+0.6.0+8caccce
2026-10-07 18:42:44,228   INFO  ==> Done (loaded 127/127)
2026-10-07 18:42:44,240   INFO  ************** EPOCH 80 EVALUATION ****************
eval:   0%|                                                
```

ReTraining Of Second Model (For Adverse Weather) To Be Mapped For Hardware Acceleration On FPGA Board (Probably Genesys2)

Problems Output Debug Console Terminal Ports

```
2026-10-16 15:30:14,519  INFO  Train:  68/80 ( 85%) [4071/5984 ( 68%)]  Loss: 0.8918 (0.896)  LR: 4.618e-04  Time cost: 1:21:10/38:08 [133:04:40/24:29:35]  Acc_iter 405000    Data time: 0.59(0.62)  Forward time: 0.46
(0.58)  Batch time: 1.05(1.20)
2026-10-16 15:31:15,539  INFO  Train:  68/80 ( 85%) [4121/5984 ( 69%)]  Loss: 0.9806 (0.896)  LR: 4.612e-04  Time cost: 1:22:11/37:08 [133:05:41/24:28:57]  Acc_iter 405050    Data time: 0.69(0.62)  Forward time: 0.55
(0.58)  Batch time: 1.23(1.20)
2026-10-16 15:32:15,908  INFO  Train:  68/80 ( 85%) [4171/5984 ( 70%)]  Loss: 0.8080 (0.896)  LR: 4.606e-04  Time cost: 1:23:11/36:09 [133:06:41/24:28:07]  Acc_iter 405100    Data time: 0.64(0.62)  Forward time: 0.55
(0.58)  Batch time: 1.19(1.20)
2026-10-16 15:33:16,322  INFO  Train:  68/80 ( 85%) [4221/5984 ( 71%)]  Loss: 0.8588 (0.896)  LR: 4.600e-04  Time cost: 1:24:12/35:09 [133:07:42/24:27:17]  Acc_iter 405150    Data time: 0.60(0.62)  Forward time: 0.53
(0.58)  Batch time: 1.13(1.20)
2026-10-16 15:34:07,927  INFO  Save latest model to /home/aniket/OpenPCDet/output/kitti_models/second_gt/v1/ckpt/latest_model
2026-10-16 15:34:18,329  INFO  Train:  68/80 ( 85%) [4271/5984 ( 71%)]  Loss: 0.7442 (0.895)  LR: 4.594e-04  Time cost: 1:25:14/34:10 [133:08:44/24:26:55]  Acc_iter 405200    Data time: 0.55(0.62)  Forward time: 0.60
(0.58)  Batch time: 1.14(1.20)
2026-10-16 15:35:17,801  INFO  Train:  68/80 ( 85%) [4321/5984 ( 72%)]  Loss: 0.7299 (0.895)  LR: 4.588e-04  Time cost: 1:26:13/33:10 [133:09:43/24:25:49]  Acc_iter 405250    Data time: 0.68(0.62)  Forward time: 0.57
(0.58)  Batch time: 1.25(1.20)
2026-10-16 15:36:16,535  INFO  Train:  68/80 ( 85%) [4371/5984 ( 73%)]  Loss: 1.020 (0.895)  LR: 4.582e-04  Time cost: 1:27:12/32:10 [133:10:42/24:24:30]  Acc_iter 405300    Data time: 0.62(0.62)  Forward time: 0.57(
0.58)  Batch time: 1.20(1.20)
2026-10-16 15:37:15,533  INFO  Train:  68/80 ( 85%) [4421/5984 ( 74%)]  Loss: 0.9129 (0.895)  LR: 4.577e-04  Time cost: 1:28:11/31:10 [133:11:41/24:23:16]  Acc_iter 405350    Data time: 0.65(0.62)  Forward time: 0.56
(0.58)  Batch time: 1.20(1.20)
2026-10-16 15:38:16,234  INFO  Train:  68/80 ( 85%) [4471/5984 ( 75%)]  Loss: 0.8820 (0.895)  LR: 4.571e-04  Time cost: 1:29:12/30:10 [133:12:41/24:22:31]  Acc_iter 405400    Data time: 0.55(0.62)  Forward time: 0.73
(0.58)  Batch time: 1.28(1.20)
```

Your results Final {after (epoch 80, val split) }

Class	Metric	Easy	Moderate	Hard
Car	3D AP_R40 @ IoU 0.70	59.3	47.9	44.5
Pedestrian	3D AP_R40 @ IoU 0.50	8.5	7.8	7.3
Cyclist	3D AP_R40 @ IoU 0.50	6.2	3.9	3.6

IoU threshold	Recall
0.3	86.6%
0.5	74.7%
0.7	44.8%

~/OpenPCDet/output/kitti_models/second/default/ckpt/checkpoint_epoch_80.pth””,
contains weights, which would be exported to genesys 2 board, PTO

Insights On Training Of Second Model To Be Mapped For Hardware Acceleration On FPGA Board (Probably Genesys2)

- Source Code Of Second Model Was Taken From Github (GitHub - open-mmlab/OpenPCDet: OpenPCDet Toolbox for LiDAR-based 3D Object Detection. · GitHub)

Weather condition	Car	Pedestrian	Cyclist
Clear weather {One of the best of LIterature}	78.62	52.98	67.16
Fog sim. [Hahner et al.] (one of the best of the literature)	77.63	45.40	55.64
Strong advection fog (one of the best of the literature)	76.69	46.92	47.73
Moderate advection fog (one of the best of the literature)	78.11	47.39	58.40
Strong spray (one of the best of the literature)	78.34	48.24	58.48

- Car class stays nearly flat across all fog types — large, dense point returns are robust to attenuation.
- Pedestrian and Cyclist degrade sharply under strong advection fog (−20 and −19 AP) — fewer points per object, more sensitive to loss.
- This is the exact robustness gap our accelerator's data path needs to tolerate — informs which classes to prioritize when validating hardware precision choices. KITTI remains our primary benchmark for this project

References : Y. Liu, Y. Tian, B. Sun, Y. Wang, F.-Y. Wang, "Parallel LiDARs Meet the Foggy Weather," IEEE J-RFID, 2022. 3D AP, moderate difficulty.

Summary

- Target confirmed: SECOND Model— a sparse-convolution, voxel-based one-stage detector well suited to hardware acceleration.
- Till Now: Project At Its Youth Stage, Worked on fog-simulation pipeline, first fog-augmented point clouds generated, And Then Training Second Model On Those Augmented Data also. (already trained on clear wheather)
- Reproduced SECOND Model baseline and layer-wise profiling to inform the accelerator dataflow.
- If Time Permits, Last But Not The Least, We May Compare Our Second Model with PointPillar Model and PointRCNN, and may also compare after training on different data, say, nuScenes is a full 360°, multi-class benchmark with a different evaluation protocol (mAP + NDS) than KITTI's per-class AP.
- Alternative under consideration: Zynq UltraScale+ (e.g. ZCU104) — adds ARM cores for pre/post-processing, at higher cost. (That Is Why Zynq not considered as Primary, So costly), and, Orange Blocks showed accuracy metrics risk points (in flow, (back)), and, Quantization and RTL design are future work

“This review establishes a verifiable path from LiDAR weather physics to a profiled software baseline, and finally to be implemented on FPGA Board (Probably Genesys 2)

Thank You!